Group 4 - Milestone 1

Ishan Shivansh Bangroo Zhi-hong Jian Tara Liu Leona Pierce

Please find the working repo here: `https://github.com/leona5139/choice_reversals`

1 Pilot design

We replicate the four treatments in Table 2 of Andreoni et al. (2020) and the instructions in Appendix C. Each player completes eight rounds and assigns ten tickets between two households for a $10 donation, given the predefined computer's allocation for each round.

In this pilot experiment on LLMs, we ran five players per model in each treatment at temperature 1.0. The full player data cover 80 players. We used Recursive Flint (for free) and GPT-Luna-6.1 (for about $0.02) through the OpenRouter API. We ran Gemini 3.1 Flash-Lite and Gemini 3.5 Flash-Lite (for free) through the Gemini API.

2 Prompts and task sequence

We use one fixed set of high-level prompts for all four treatments (Appendix B). The sequence of messages may differe across treatment, see below. Each player starts a fresh message thread. Before each API call, we add the information revealed so far and the current question. We send the full thread and append the reply with `"role": "assistant"` (see the player data examples).

We asked each model to return its allocation as a JSON object. We parsed the text the models returned and found no formatting errors.

Each of the four main treatments has a different sequence of the three tasks below.

- **A.** The player allocates ten tickets. We then draw the winning ticket and reveal which group holds it.
- **P.** We draw the ticket and reveal its winning group first. The player allocates if their group wins. We then reveal the ticket number.
- A^R**.** The player submits all four allocations before any group is revealed. We then reveal the group one round at a time. When the player's group wins, they can keep or change that round's allocation.

All four treatments end with four A^R rounds. Rounds 1–4 are all A (`4A_4AR`), all P (`4P_4AR`), two A followed by two P (`2A2P_4AR`), or two P followed by two A (`2P2A_4AR`).

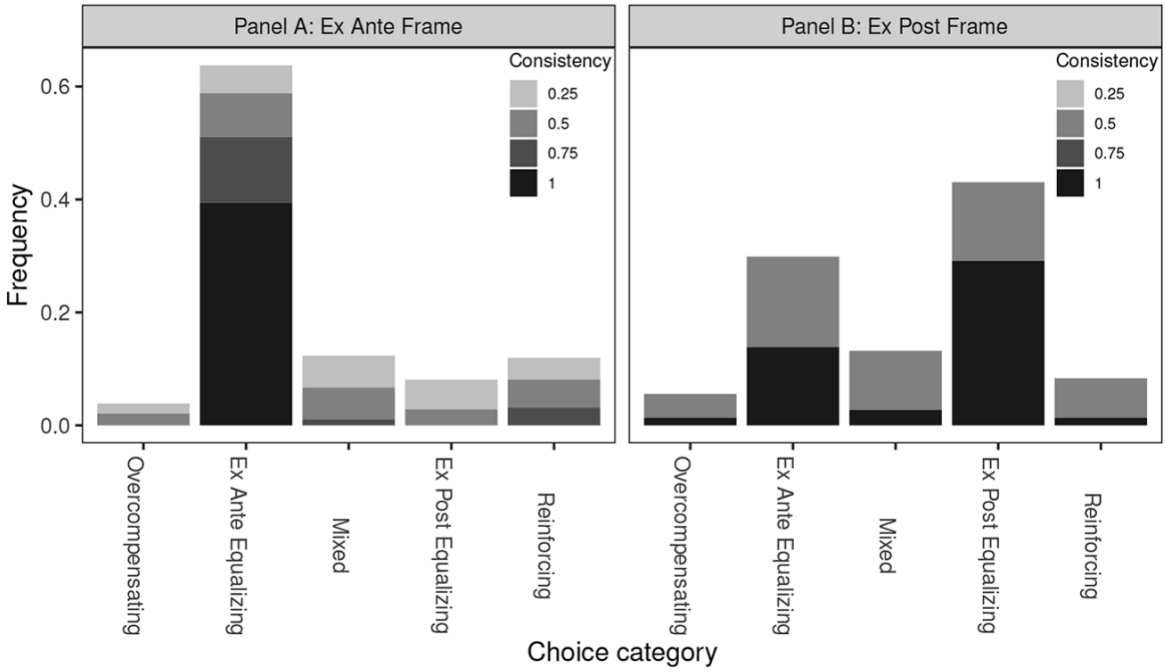
3 Finding

Figure 1 compares human and LLM choices in rounds 1–4. In the ex ante frame, players allocate before the draw. In the ex post frame, they allocate after learning their group won.

- Even in the ex ante frame, models chose the 5/5 split in 39 of 80 choices (49%), compared with about 8% for humans.
- Choices varied by model (Figure 3). GPT-Luna-6.1 chose the ex ante equalizing allocation in all 20 choices in the ex ante frame and 9 of 10 in the ex post frame, which is closer to what we see from the human subjects. Gemini 3.5 Flash-Lite is particularly unresponsive to the computer's allocation and sticks to 5/5.

In rounds 5–8 of the treatment that starts with four ex ante rounds, only GPT-Luna-6.1 revised. It switched from the ex ante equalizing allocation to 5/5 in 9 of its 10 revision opportunities (Figure 2 and Figure 4).

Human experiment (Andreoni et al., 2020, fig. 1)



LLM pilot

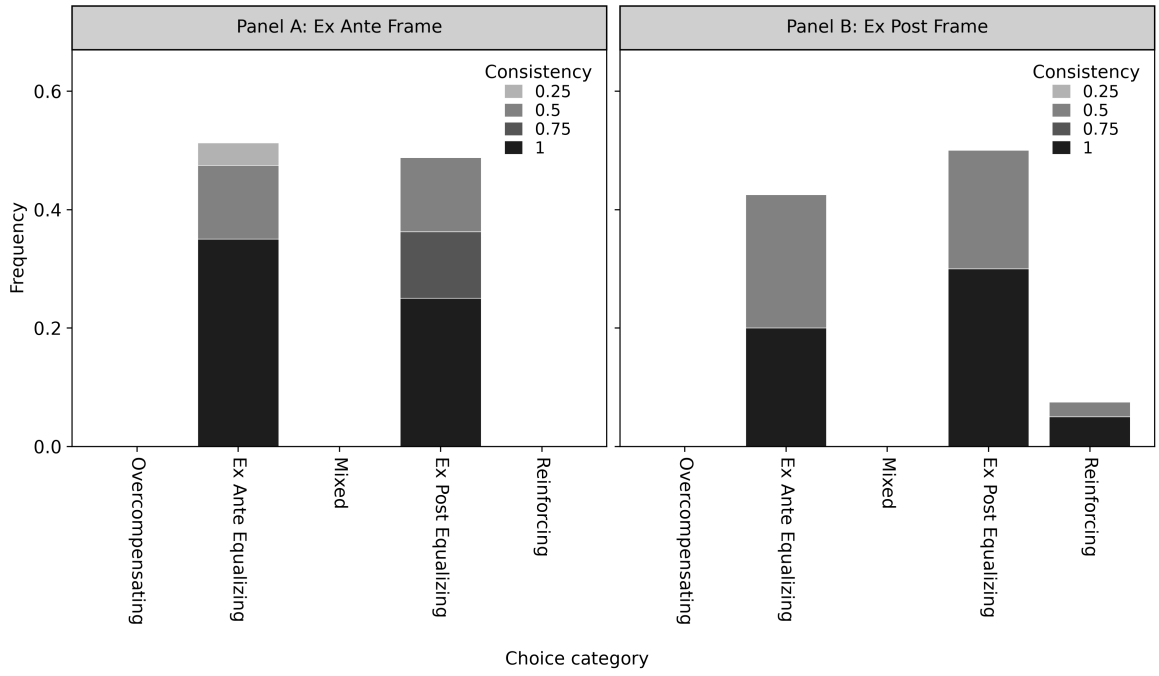
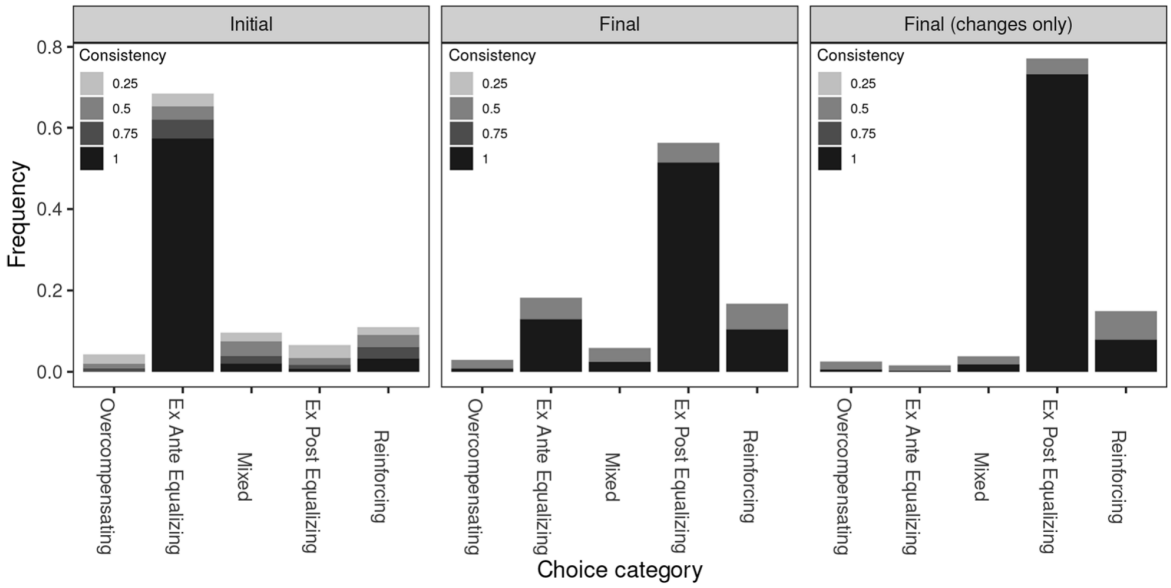


Figure 1: Allocations in A and P tasks, rounds 1–4.

Human experiment (Andreoni et al., 2020, fig. 3)



LLM pilot

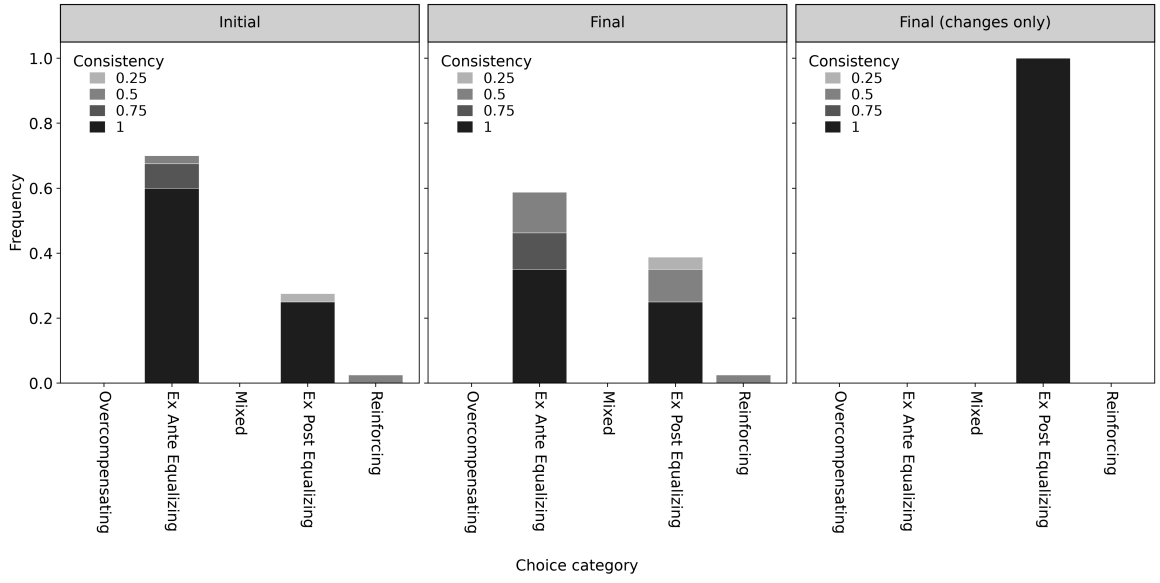


Figure 2: Initial and final allocations in 4A_4AR, rounds 5–8.

A Pilot results by model

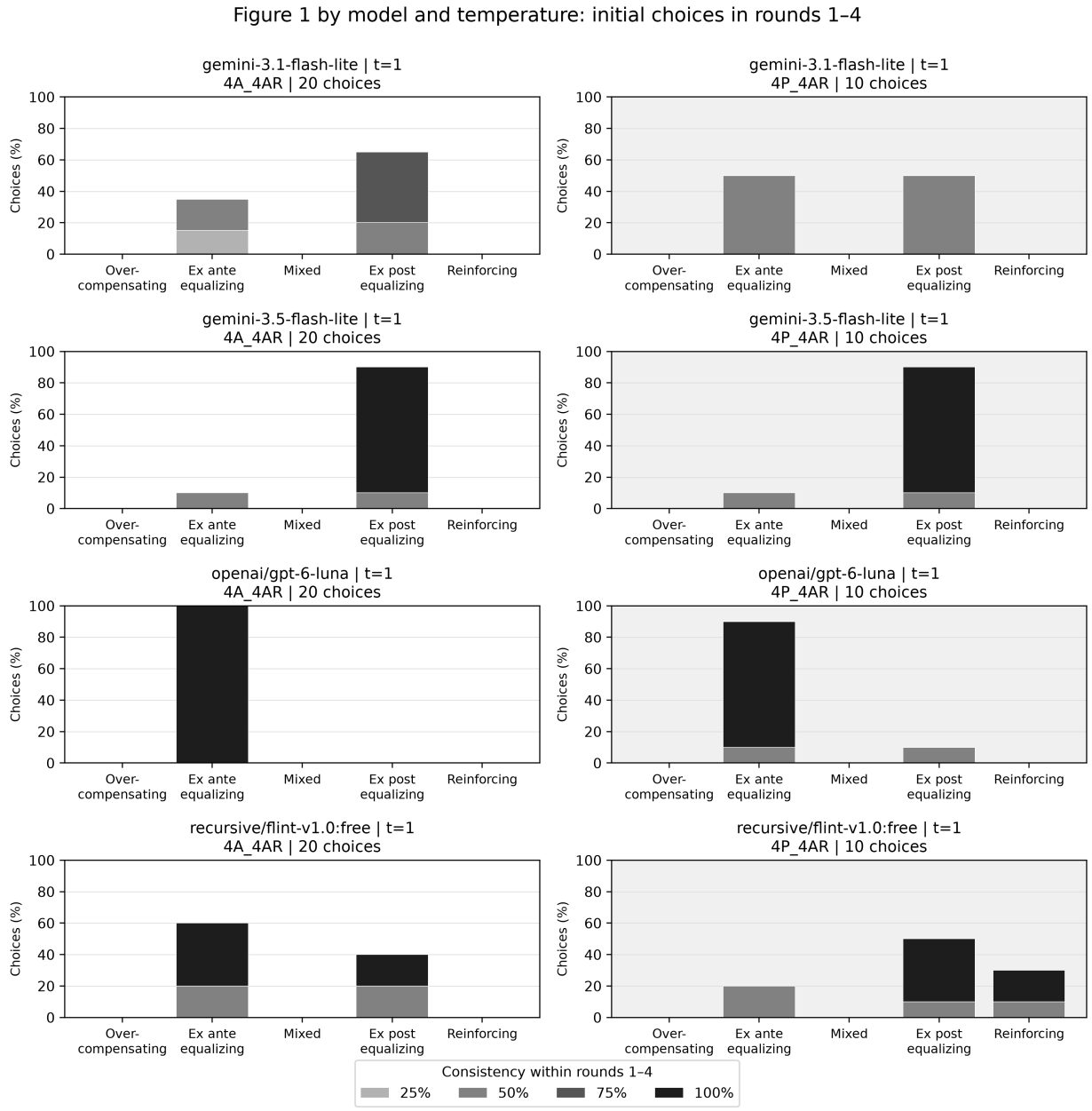


Figure 3: Initial allocations in A and P tasks by model.

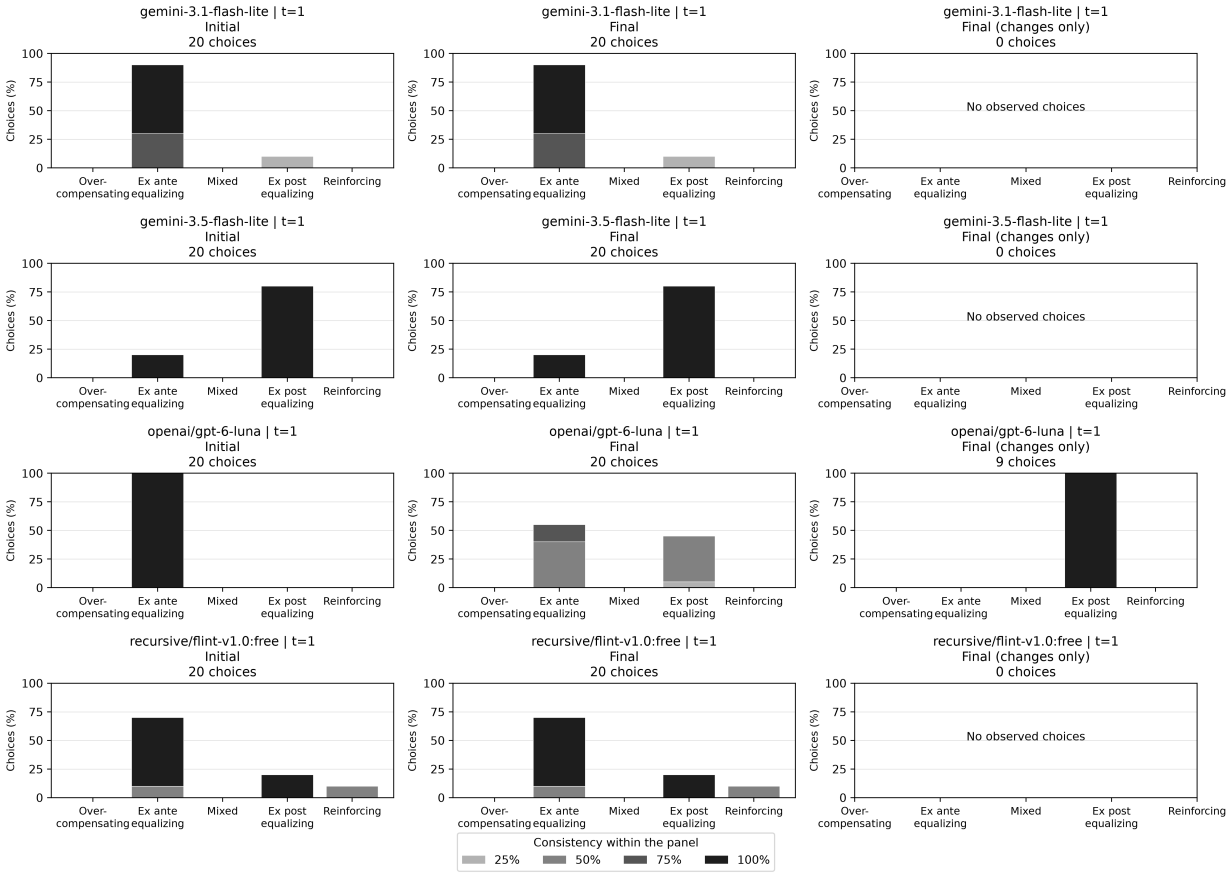


Figure 4: Initial and final allocations in 4A_4AR by model, rounds 5–8.

B Pilot prompt

```
[SYSTEM]
You are participating in study of allocation behaviors. Follow the instructions for each round and
  make your own decisions.

[INSTRUCTIONS]
In this study, you will decide how to allocate chances to win $10 donations among actual
  households in Kenya.

There are eight rounds. Each round concerns two different recipient households, called Household A
  and Household B. You will not be given individual information that distinguishes A from B or
  indicates that one is more deserving of a donation.

Your decisions concern tickets 1-10, which can be assigned between A and B. The computer has
  already assigned tickets 11-20. You will see the computer's allocation for that round before
  making any allocation decision, and you cannot change it. One of the 20 tickets is selected at
  random for each round as the winning ticket: the household holding that ticket is the winner for
  that round.

Only one of the eight rounds will actually count. After all eight rounds are complete, one round
  will be selected at random, and a $10 donation will be made to that round's winning household.
  Treat each round as one that could count.

[ALLOCATION]
Round {round}. In this round, the computer has already assigned {computer_a} of its 10 tickets to
  Household A and {computer_b} to Household B. Its allocation cannot be changed.

Assign your 10 tickets between Household A and Household B based on the information you have
  received for this round.

[GROUP_REVEAL]
For this round (Round {round}), the winning ticket has been put in the {group} set of 10 tickets
  (your tickets are 1-10; computer tickets are 11-20). Its specific number has not been revealed.

If it is in the computer set, the computer's existing allocation determines the winner and you
  have no further allocation decision for this round. If it is in your set, your allocation of
  tickets 1-10 can still determine which household wins.

[REVISION]
Earlier in Round {round}, you assigned {choice_a} of your tickets to Household A and {choice_b} to
  Household B. The computer assigned {computer_a} tickets to A and {computer_b} to B. You now know
  that the winning ticket is one of your tickets 1-10, but you do not know its specific number.

Review your earlier allocation. You may keep it or change it. Submit the allocation of your 10
  tickets that you want to use for this round. To keep your earlier choice, submit the same
  numbers again.

[OUTCOME_A]
Round {round} is complete. The winning ticket is in the {group} set of tickets.

[OUTCOME_P]
Round {round} is complete. The winning ticket is ticket {ticket}, in the {group} set of tickets.

[RESPONSE_FORMAT]
Respond with a single JSON object and nothing else: {"A": <number of your 10 tickets to Household
  A>, "B": <number of your 10 tickets to Household B>}. Use whole numbers from 0 to 10 that sum to
  10.
```